

Lecture 34: Caching and Distributed Systems

Caching, Distributed Caching, Cache Invalidation

& Eviction

1. Why Do We Need Caching?

Suppose our backend receives:

```
GET /user/123
```

Without caching:

```
Client
  |
  v
Backend
  |
  v
Database
  |
  v
Return Response
```

If 100,000 users request the same data, the database may receive 100,000 queries.

100,000 Requests

|

v

Backend

|

v

100,000 DB Queries

But much of this data may be identical.

For example:

```
user:123
{
  name: "Rohit",
  city: "Delhi"
}
```

Instead of repeatedly asking the database, we can store a copy somewhere faster.

That copy is called a cache.

2. What Is Caching?

Caching means:

Store frequently accessed data in a faster storage layer so we don't have to repeatedly fetch or compute it from the original source.

Architecture:

```
Client
  |
  v
Backend
  |
  v
Cache
  |
  v
Database
```

Suppose the cache is Redis.

First Request

```
GET user:123
```

Redis:

```
MISS
```

Then:

```
Backend
  |
  v
Redis
  |
  | MISS
  v
Database
```



Then return the response.

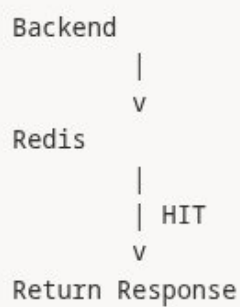
Second Request

```
GET user:123
```

Redis already contains:

```
user:123 → Rohit
```

So:



Database is not involved.

3. Why Is Cache Faster?

Databases generally need to handle:

- Disk access
- Indexes
- Query execution
- Joins
- Locks

- Transactions
- Network calls

A cache such as Redis generally keeps data in memory.

So:

- RAM access
- 1
- Very fast

Therefore caching helps us reduce:

- Latency ↓
- Database Load ↓
- CPU Usage ↓
- Network Calls ↓
- Cost ↓

4. Cache Hit and Cache Miss

Cache Hit

Data is available in cache.



Example:

Redis:
user:123 → Rohit

Request:

GET user:123

Result:

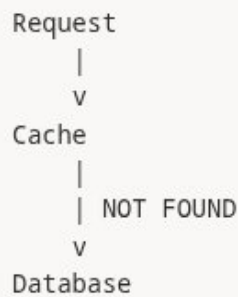
CACHE HIT

Plain Text



Cache Miss

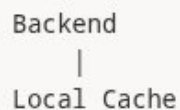
Data is not available.



Then we commonly cache the database response.

5. What Is Distributed Caching?

Initially suppose we have one backend server.



We could even do:

```
const cache = new Map();
```

This works for one server.

But now we scale the backend.

```
      Load Balancer
    /   |
  /   |
Server 1 Server 2 Server 3
```

Each server has separate RAM.

Therefore:

```
Server 1 Cache
user:123 → Rohit
Server 2 Cache
EMPTY
Server 3 Cache
EMPTY
```

Suppose the first request goes to Server 1.

Server 1 caches:

```
user:123
```

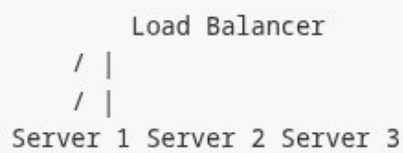
The next request may go to Server 2.

Server 2 doesn't know what Server 1 cached.

Therefore Server 2 again queries the database.

Solution: Shared External Cache

Move the cache outside individual backend servers.



Now all backend servers share the same cache.

Server 1:

```
Redis MISS
  ↓
Database

Cache user:123
```

Later Server 3 receives the same request:

```
Server 3
  |
  v
Redis
  |
  | HIT
  v
Response
```

Server 3 benefits from data cached by Server 1.

This is the basic idea behind distributed caching.

Popular technologies:

```
Redis
Memcached
```

6. Cache Itself Can Also Be Distributed

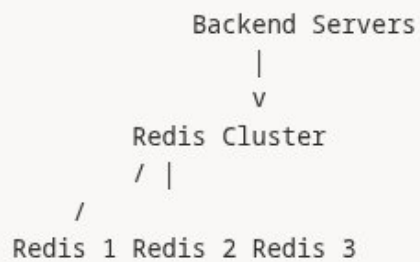
Suppose our cache grows too large for one Redis machine.

Example:

```
Redis Memory = 64 GB
Required Cache = 500 GB
```

One Redis server cannot hold everything.

So Redis itself can be distributed.



Data can be sharded.

For example:

```
user:1 → Redis 1
user:2 → Redis 2
user:3 → Redis 3
```

So distributed caching can mean:

Multiple backend servers sharing an external cache

and at larger scale:

The cache itself is distributed across multiple machines

7. CDN vs Redis Cache

Both are caches, but they are usually solving different problems.

User
|

 v
CDN
 |
 v
Backend
 |
 v
Redis
 |
 v
Database

CDN

Usually caches content closer to users.

Examples:

Images
Videos
CSS
JavaScript
Fonts
HTML

CDN is a geographically distributed cache.

Redis

Usually caches application data closer to the backend.

Examples:

```
User profile
Product information
Session
API response
Recommendations
Rate limiting counters
```

So remember:

```
CDN
= cache near users
```

```
Redis
= cache near backend
```

8. What Is a Caching Strategy?

Now suppose we have:

```
Backend
  |
Redis
  |
Database
```

We need to answer:

How should data enter the cache?

Who checks the cache?

What happens during writes?

These decisions are called caching strategies.

Major strategies:

```
Cache Aside
Read Through
Write Through
Write Behind / Write Back
```

9. Cache-Aside Strategy

Cache-aside is one of the most commonly used caching strategies.

The application itself manages the cache.

Suppose:

```
GET /product/100
```

Backend first checks Redis.

```
Backend
  |
  v
Redis
```

If HIT:

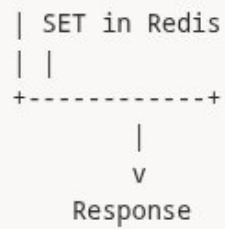
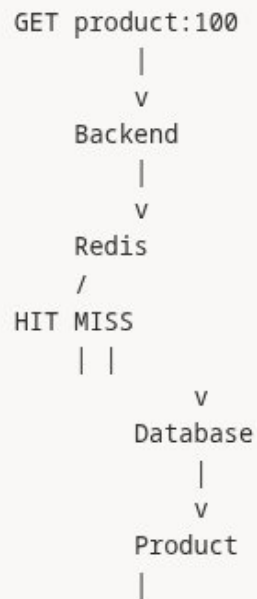
```
Redis
  |
  | HIT
  v
Return Product
```

If MISS:

```
Redis
  |
  | MISS
  v
Database
```

Backend retrieves the product and places it in Redis.

Complete flow:



Example:

```
let product = await redis.get("product:100");
if (!product) {
  product = await database.getProduct(100);
  await redis.set(
    "product:100",
    JSON.stringify(product),
    { EX: 300 }
  );
}
return product;
```

The application is responsible for:

```
Checking Redis
Fetching DB
Updating Redis
```

Hence the name:

Cache is sitting “aside” from the database.

10. Read-Through Cache

Read-through looks similar to cache-aside, but responsibility changes.

Backend asks:

```
Give me product:100
```

The caching layer itself handles the miss.

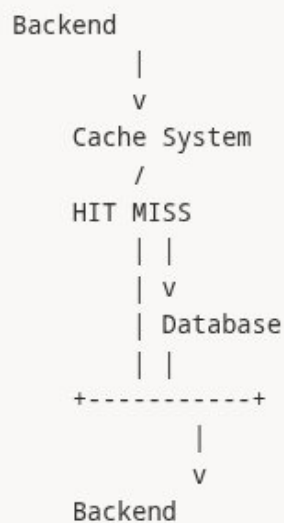
```
Backend
  |
  v
Cache Layer
  |
  | MISS
  v
Database
```

Backend doesn't explicitly write:

```
if cache miss:
    query database
```

That logic is handled by the caching infrastructure/library.

Conceptually:



Important:

Plain Redis does not automatically know how to query MongoDB/MySQL.

We need an additional library or caching layer to implement this abstraction.

11. Write-Through Cache

Now let's think about writes.

Suppose:

```
User changes name:  
Rohit  
  ↓  
Rohit Negi
```

With write-through:

```
Application  
  |  
  v  
  Cache  
  |  
  v  
Database
```

Both cache and database are updated.

Conceptually:

```
UPDATE user:123  
  |  
  v  
  Cache  
  |  
  v  
Database
```

Now

Now:

```
Cache = latest value  
Database = latest value
```

Advantage:

```
Cache remains fresh
```

Disadvantage:

```
Every write involves cache + database
```

So writes may become more expensive.

12. Write-Behind / Write-Back Cache

Now suppose we want extremely fast writes.

Instead of immediately updating the database:

```
Application  
  |  
  v  
Cache
```

Return success.

Later:

```
Cache  
  |  
  | asynchronously  
  v  
Database
```

Example:

```
User likes a YouTube video
Current Likes:
10,000
User clicks LIKE
Redis:
10,001
Return immediately
```

Later:

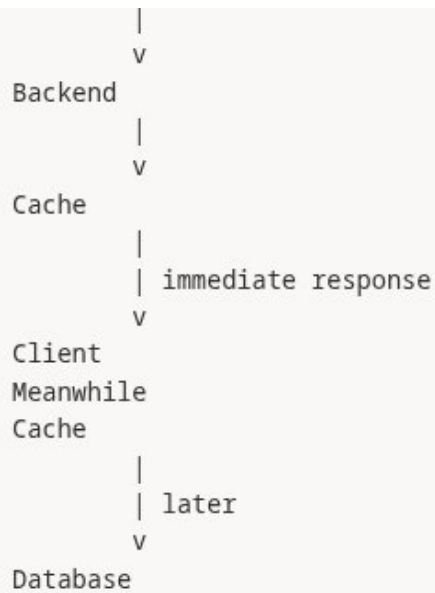
```
Redis
  |
  v
Database
```

This is called:

```
Write Back
or
Write Behind
```

Architecture:

```
Request
```



Benefit:

Very fast writes

Problem:

Suppose cache crashes before writing to the DB.



Data could be lost.

Therefore this strategy requires much more careful durability design.

13. Comparison of Caching Strategies

Cache Aside

App manages cache manually

Very common

Good for read-heavy systems

Read Through

Cache layer manages DB loading

Application code becomes simpler

Write Through

Write cache + DB together

Better consistency

Slower writes

Write Behind

Write cache first

DB updated later

Very fast writes

Higher data-loss risk

14. The Cache Invalidation Problem

Caching introduces a new problem.

Suppose Redis contains:

```
user:123  
{
```

```
    name: "Rohit",  
    city: "Delhi"  
}
```

Database also contains:

```
city = Delhi
```

Everything is correct.

Then user changes city:

```
Delhi  
  
Bangalore
```

Database becomes:

```
city = Bangalore
```

But Redis still contains:

```
city = Delhi
```

Now:

```
Database  
Bangalore  
Cache  
Delhi
```

The cache contains stale data.

This is called the cache invalidation problem.

The fundamental question is:

When the original data changes, how do cached copies learn that their data is no longer valid?

15. Solution 1 - TTL

The simplest solution is expiration.

Example:

```
user:123  
TTL = 5 minutes
```

After 5 minutes:

```
Cache entry expires
```

Next request:

```
Cache MISS
  |
  v
Database
  |
  v
Latest data
```

Advantage:

Very simple

Problem:

Suppose data changes after only 10 seconds.

TTL = 5 minutes

Users may see stale data for almost:

4 minutes 50 seconds

So TTL provides eventual freshness, not immediate freshness.

16. Solution 2 - Explicit Cache Invalidation

Suppose we update the database.

```
UPDATE user:123
```

Immediately delete the cache entry:

```
DELETE user:123
```

Flow:

```
graph TD
    A[User updates profile] --> B[Backend]
    B --> C[Update Database]
    C --> D[Delete Redis Key]
```

Now next request:

```
GET user:123
```

Redis:

MISS

Therefore:

```
Redis MISS
  |
  v
Database
  |
  | latest value
  v
Redis
```

Cache becomes fresh again.

17. Delete Cache or Update Cache?

Suppose DB changes from:

Delhi → Bangalore

We have two choices.

Option 1:

```
Update Database  
Update Redis
```

Option 2:

```
Update Database  
Delete Redis
```

Often deleting is safer.

Why?

Because cached data might not simply be:

```
user
```

Maybe cached data was generated from:

```
User table  
+  
Orders table  
+  
Subscription  
+  
Permissions  
+  
Recommendation
```

Correctly updating this derived object can become complicated.

Instead:

```
Delete cached object
```

Next request reconstructs it from the latest source.

Therefore a common pattern is:

```
UPDATE DATABASE  
  t  
DELETE CACHE
```

18. Why Cache Invalidation Is Difficult

At first it sounds easy:

```
Update DB  
Delete cache
```

But concurrency creates problems.

Suppose two servers:

```
Server A  
Server B
```

Both operate simultaneously.

Imagine:

```
Step 1  
Server B reads OLD value from DB  
Step 2  
Server A updates DB with NEW value  
Step 3  
Server A deletes cache
```

Step 4

Server B stores the OLD value into Redis

Final state:

Database = NEW

Redis = OLD

Even though Server A correctly invalidated the cache.

Diagram:

```
Server A Server B
  |   |
  |   | Read Old DB
  |   |
  |   |
Update DB
  |
Delete Cache
  |
  |
  |
  v
  v
  v
Database = NEW
Cache = OLD
```

This is why cache invalidation becomes difficult in distributed systems.

The problem isn't:

How do I run DEL in Redis?

The problem is:

How do I keep multiple copies of changing data consistent when many operations happen concurrently?

19. Cache Stampede / Thundering Herd

Caching has another problem.

Suppose:

```
product:iphone  
TTL expires at 10:00 PM
```

Exactly at 10:00 PM:

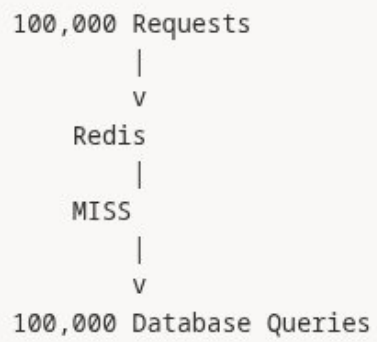
```
100,000 users
```

request the same product.

Everyone sees:

```
CACHE MISS
```

A bad system:



Database can get overloaded.

This is called:

Cache Stampede

or:

Thundering Herd

Possible solutions include:

Distributed locking
Request coalescing

Stale-while-revalidate
TTL jitter

20. TTL Jitter

Suppose we cache 1 million products.

If every key has:

TTL = exactly 5 minutes

and all were inserted around the same time:

5 minutes later

many keys expire together.

Huge DB spike.

Instead:

TTL = 5 minutes + random variation

Examples:

```
product:1 → 282 seconds  
product:2 → 316 seconds  
product:3 → 301 seconds  
product:4 → 274 seconds
```

Now expiration is spread over time.

This randomness is called:

TTL Jitter

21. What Is Cache Eviction?

Cache invalidation and eviction are completely different.

Suppose Redis has:

Memory = 1 GB

And all 1 GB is already used.

Now you want to insert:

product:999

Redis needs memory.

Something may have to be removed.

The question becomes:

Which cached item should be removed?

This is called cache eviction.

Example:

```
Cache
A
B
C
D
E
Memory Full
New Item:
F
```

We need:

```
Remove ?
```

That decision is made using an eviction policy.

22. LRU - Least Recently Used

LRU asks:

Which item has not been used for the longest time?

Example:

```
A → accessed 1 second ago
B → accessed 10 seconds ago
C → accessed 2 minutes ago
D → accessed 1 hour ago
```

If memory is full:

```
Remove D
```

because it was least recently used.

Mental model:

```
Recent access = likely needed again  
Old access = safer to remove
```

23. LFU - Least Frequently Used

LFU asks:

Which item has been used the fewest number of times?

Example:

```
A → 10,000 requests  
B → 8,000 requests  
C → 200 requests
```

D → 2 requests

Remove:

D

because D is least frequently used.

Mental model:

Popular objects should stay cached

24. LRU vs LFU

Consider:

Product A
Requested 100,000 times historically
Last request:
1 hour ago
Product B
Requested only 2 times
Last request:
1 second ago

LRU says:

Remove Product A
because A wasn't used recently.

LFU says:

Remove Product B
because B is not popular.

Therefore:

LRU
"When was it last used?"

versus:

LFU
"How often was it used?"

25. FIFO - First In First Out

Very simple strategy.

Remove the item that entered cache earliest.

Example:

```
Inserted order:
```

```
A
```

```
B
```

```
C
```

```
D
```

Memory full.

Insert E.

Remove:

A

FIFO doesn't care whether A is extremely popular.

That's why it can be inefficient for many workloads.

26. Random Eviction

Another simple strategy:

```
Pick random key
```

```
Delete it
```

Not intelligent, but extremely cheap.

Can be useful in some workloads where sophisticated tracking isn't worth the overhead.

27. Expiration vs Eviction

This distinction is extremely important.

Expiration

TTL finishes.

Example:

```
user:123  
TTL = 5 minutes
```

After 5 minutes:

```
Remove user:123
```

Reason:

```
TIME
```

Eviction

Cache memory is full.

Example:

```
Redis memory = 1 GB  
Used memory = 1 GB
```

Need to insert new key.

Redis may remove another key.

Reason:

```
MEMORY PRESSURE
```

Therefore:

```
Expiration  
  =  
Eviction
```

An item could have:

```
TTL remaining = 30 minutes
```

but still get evicted if Redis is out of memory and the configured policy allows it.

28. Invalidation vs Eviction

Another important distinction:

```
Invalidation
```

means:

Data is no longer correct.

Example:

```
Database changed
```

Therefore cached copy should be removed/updated.

Eviction

means:

Cache needs space.

The data might still be perfectly correct.

But we remove it because memory is limited.

Therefore:

```
Invalidation  
→ correctness problem  
Eviction  
→ memory management problem
```

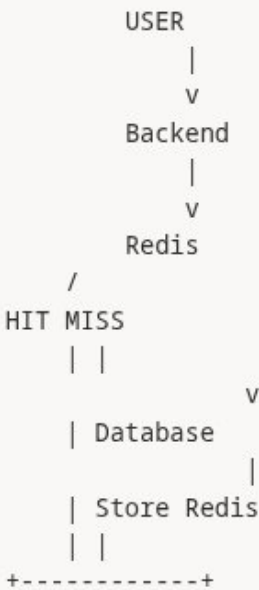
29. Complete Picture

Suppose we're building Amazon.

User requests:

GET /product/iphone

Architecture:



I

v

Response

Now suppose product price changes:

₹80,000

₹75,000

We do:

```
Database Update
  |
  v
Delete Redis Key
```

This is:

CACHE INVALIDATION

Next request rebuilds the cache.

Later Redis memory becomes full.

Redis needs to insert:

product:macbook

It chooses some old/unpopular cache key and removes it.

This is:

CACHE EVICTION

Later:

product:iphone

reaches its 10-minute TTL and disappears.

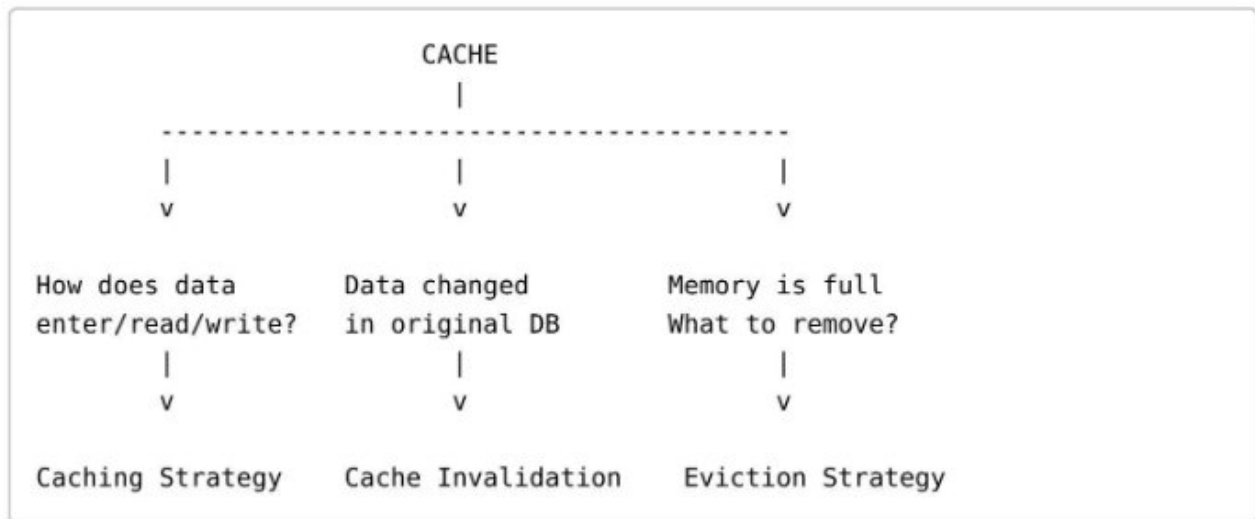
This is:

CACHE EXPIRATION

These are three separate events.

30. Final Mental Model

Keep this diagram in your head:



Caching Strategy examples:

Cache Aside
Read Through
Write Through
Write Behind

Invalidation approaches:

TTL
Explicit Delete
Explicit Update
Versioning
Events / Pub-Sub

Eviction strategies:

LRU
LFU
FIFO

31. One-Line Definitions for Interviews

Cache

Temporary fast storage used to avoid repeatedly fetching or computing the same data.

Distributed Cache

A cache shared by multiple application servers, often itself spread across multiple machines.

Cache Hit

Requested data is found in cache.

Cache Miss

Requested data is not present in cache.

Cache-Aside

Application checks cache first and loads from database on a miss.

Cache Invalidation

Removing or updating cached data when the original data changes.

Cache Expiration

Automatically removing cached data after its TTL finishes.

Cache Eviction

Removing cached data because cache memory is running out.

LRU

Remove the least recently accessed item.

LFU

Remove the least frequently accessed item.

Cache Stampede

Many requests simultaneously miss the same cache entry and overload the database.

32. Most Important Principle

Caching improves performance by creating additional copies of data.

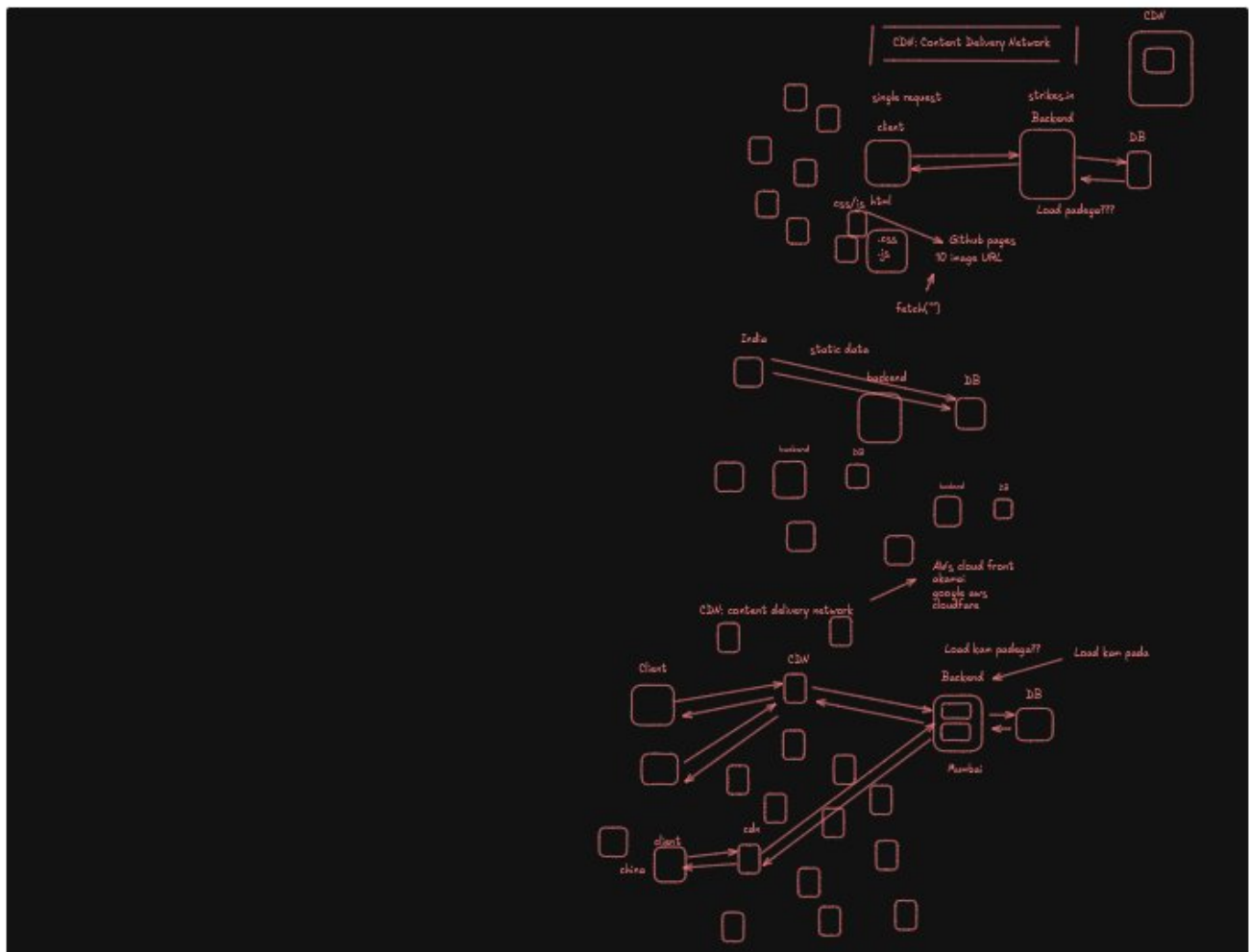
But the moment we create additional copies, we introduce a new problem:

```
One Original Data
  caching
Multiple Copies
```

Now we must answer:

What happens when thousands of requests miss together?

The faster we want our system to become using cached copies, the more carefully we need to manage those copies.





→ 1 piece data: 1 lakh user
data - 1 lakh

CDN: html,css,js, image, videos (Data) → Same hoge
→ Dynamic Data (not store)

1000 cdn server?

client: rohit

rohit → back information change kari hai



React → npm run build

script.js
style.css

new file

client
index.html

CDN

backend

strikesin
index.html tit 5 min
css/peasde → 1 year
js 1 year → strikesin → 1 year
images: 1 year
user: Video delete maar di
/rohitnegultra3

comment like

client

CDN

backend

client

comment like
yt video

https://strikesin.com/video.mp4

TTL: 30 min

origin: strikesin: 19.8.7.6

public key

Certificate: strikesin

key/value

client

cdn

Backend

mq

private network

key/value

public key

private key

key/value

strikesin public key

CA

Aug UI

strikesin public key

CA

key/value

vinat kahi 26

Redis

Database

backend server

